

Системні виклики



©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Апаратне розмежування привілей

- Більшість сучасних процесорів, включаючи вбудовувані, містять засоби розмежування користувацького та привілейованого коду¹.
- Хоча реалізації сильно відрізняються, загальна ідея наступна. Є мінімум два режими роботи CPU: привілейований (supervisor або kernel mode) та користувацький (user mode).
- ◊ В привілейованому режимі можна виконувати будь-які операції.
- ◊ В користувацькому заборонене все, що може не дати акуратно написаній ОС убезпечити себе та інші процеси від некоректної поведінки одного із них.
- Приклад: на x86 команди `mov CR0, reg`, де CR0 – головний регістр керування режимом CPU, чи `LIDT` – завантажити Interrupt Descriptor Table Register, дозволена лише в (найбільш) привілейованому режимі.
- ◊ Також, сторінки пам'яті помічені бітом U/S – User/Supervisor, і останні доступні лише в привілейованому режимі, навіть якщо відображені в адресний простір процесу.
- Деякі подробиці реальних систем – в кінці.

¹Є вони і в x86, і в ARM Cortex M, включаючи мікроконтролери сімейства STM32, а от AVR їх не має. ▶

Системні виклики

- В ОС із монолітним ядром² код ядра, код, що працює в привілейованому режимі CPU, разом із його віртуальною пам'яттю та іншими ресурсами, називають kernel space, а користувацький, відповідно – user space.
- ◇ Поведінка та термінологія мікроядерних³ та екзоядерних ОС може відрізнитися.
- ◇ Не плутайте kernel mode/space із правами адміністратора, що надаються окремим користувачам⁴. Процеси такого користувача, як і всіх інших, виконуються в user space, але ОС надає йому більше прав – більше його запитів буде виконано успішно.
- Однак, користувацькому коду потрібен доступ до можливостей, реалізованих ядром.
- Системні виклики — спосіб надати цей доступ. При цьому, відбувається перевірка, чи користувач має відповідні права.

²Класифікацію ОС ми вивчатимемо трохи пізніше. Зараз достатньо знати, що Windows, Linux, включаючи Android, FreeBSD є системами з монолітним ядром. Ядро Windows NT іноді класифікують як гібридне.

³Приклади: MINIX, GNU Hurd, L4. Apple XNU – ядро MacOS є гібридним, зібраним із Open Source Foundation Mach та FreeBSD.

⁴Наприклад, root чи учасники групи sudoers під Unix

Бібліотечні виклики

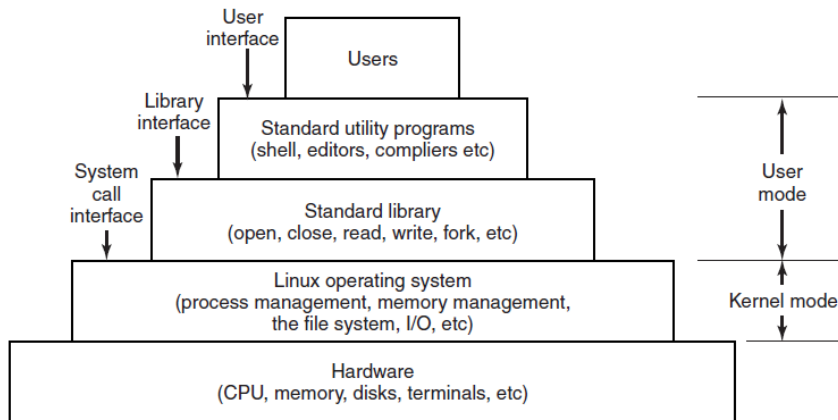
- Часто, користувацьким програмам надаються (більш чи менш зручні) обгортки над системними викликами — функції із системних бібліотек.
- В Unix-подібних системах це, зазвичай, відносно тонкі обгортки поверх системних викликів.
- ◊ Головною такою бібліотекою є `libc`.
- Під Linux, системні виклики, для яких немає безпосередньої обгортки⁵, можна викликати із високорівневого коду⁶ за допомогою функції `syscall()`⁷.
- Для Windows NT ситуація радикально відрізняється — базовий "публічний" API, Win32 API, фактично, є обгорткою над Native API, які не документовані, але є значно ближчі до власне системних викликів.
- Головне призначення таких обгортки-wrapper-ів — позбавити програміста потреби безпосередньо взаємодіяти із механізмом системних викликів. Це зменшує кількість помилок та збільшує портабельність (переносимість) програм.

⁵Наприклад, `ctmp` – порівняти спільні ресурси двох процесів Linux.

⁶Як альтернатива описаним нижче асемблерним вставкам.

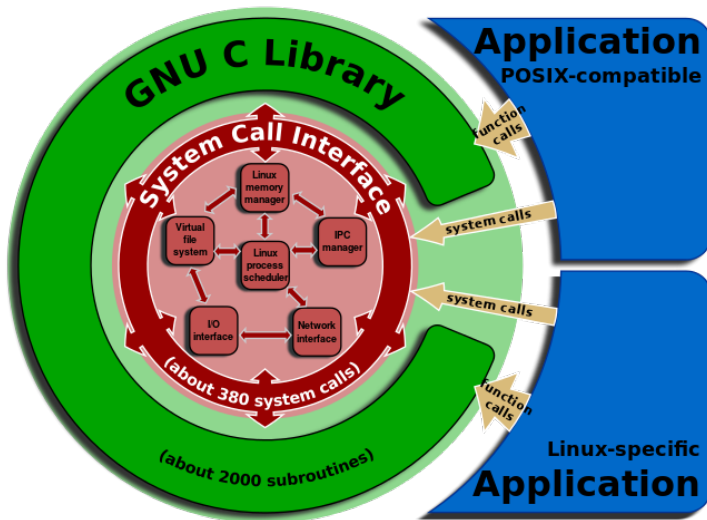
⁷<http://man7.org/linux/man-pages/man2/syscall.2.html>

Структура інтерфейсів Linux

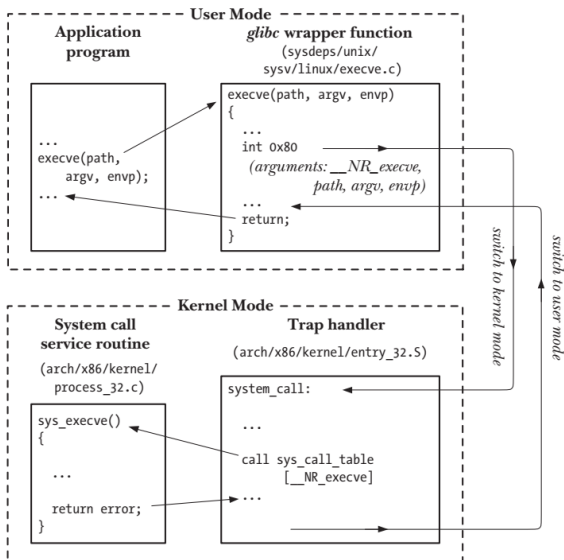


- Взято із Tanenbaum, Bos, "Modern Operating Systems", 4th ed.

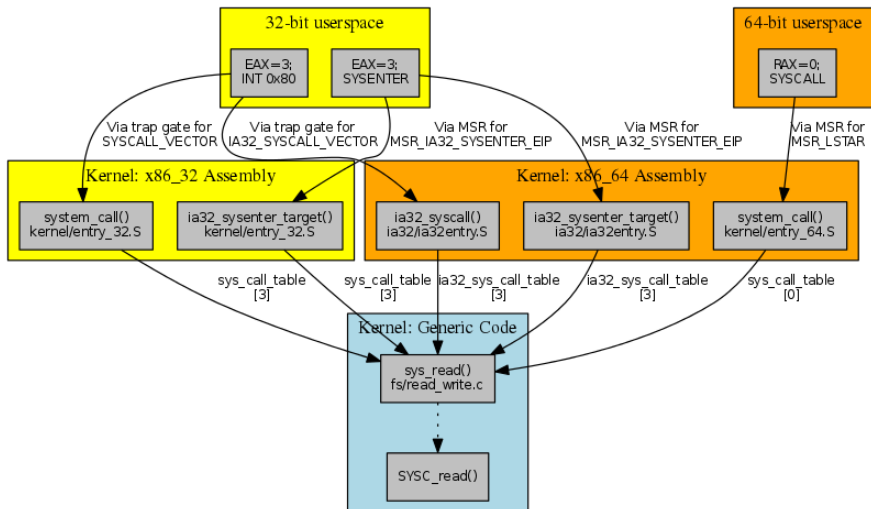
Ще одна гарна картинка структури інтерфейсів Linux



Реалізація системних викликів



- Ілюстрація з книги Керріска "Програмні інтерфейси Лінукс"



- Жовте – 32-бітовий код (зокрема – і user-space), помаранчеве – 64-бітовий.
- Взято тут: <https://lwn.net/Articles/604515/>, "Anatomy of a system call, part 2".

- Виклик може бути ініційований перериванням чи **пасткою** (trap). Наприклад, DOS використовував переривання із номером 21h, а Linux — 80h.
- Сучасні архітектури, зазвичай, мають спеціальні команди системного виклику:
 - ◇ SYSCALL/SYSRET – x86-32 від AMD, x86-64 всі.
 - ◇ SYSENTER/SYSEXIT – x86-32 від Intel⁸.
 - ◇ SVC (SerVice Call) – ARM Cortex
 - ◇ SVC і PC – мейнфрейми IBM,
 - ◇ EPC (Enter Privileged Code) – Itanium, IA-64 тощо.
- Існує також всіяка екзотика, типу call gate в x86 чи Multics/Honeywell 6180.
- Курйоз: на 386 найшвидшим способом системного виклику із V86 була некоректна інструкція: <https://devblogs.microsoft.com/oldnewthing/20041215-00/?p=37003>, "The hunt for a faster syscall trap".
- Враховуючи, що поняття пастки, trap, викликало багато питань, див.:
<https://stackoverflow.com/questions/3149175/what-is-the-difference-between-trap-and-interrupt/37558741> та
[https://en.wikipedia.org/wiki/Trap_\(computing\)](https://en.wikipedia.org/wiki/Trap_(computing)).

⁸Історія від розробника: https://web.archive.org/web/20120825073036/http://semipublic.comp-arch.net/wiki/SYSENTER/SYSEXIT_vs._SYSCALL/SYSRET із подробицями, чому вона така. ▶

Приклад – Linux, x86-32, int 80h, синтаксис Intel, діалект GNU as

```
.intel_syntax noprefix # Same as: -msyntax=intel -mnaked-reg
.global _start
```

```
.text
```

```
_start:
```

```
    mov eax, 4                # __NR_write (sys_write) == 4
    mov ebx, 1                # File handler of stdout
    mov ecx, OFFSET msg      # Non-PIC код
    mov edx, OFFSET msg_size
    int 0x80
```

```
    mov eax, 1                # __NR_exit (sys_exit) == 1
    xor ebx, ebx              # exit code 0
    int 0x80
```

```
.data
```

```
msg:
```

```
.ascii "Hello 32-bit Linux world!\n"
```

```
msg_size = . - msg
```

- Детальніше про `int 0x80`:
- Згідно коду із ядра Linux для x86-32, номери відповідних викликів такі:

```
#define __NR_exit 1  
#define __NR_write 4
```
- Номер системного виклику – в EAX.
- В EBX, ECX, EDX, ESI, EDI, EBP – аргументи з першого по шостий.
- Прототип системного виклику на C
(<http://man7.org/linux/man-pages/man2/write.2.html>):

```
ssize_t write(int fd, const void *buf, size_t nbytes);
```

Порівняйте:

```
mov    eax, 4          # __NR_write == 4 -- запис у файл  
mov    ebx, 1          # Дескриптор для stdout  
mov    ecx, OFFSET msg # Вказівник на буфер  
mov    edx, OFFSET msg_size # Розмір буфера  
int    0x80
```

- Знайти список системних викликів можна в коді ядра, десь тут:
`$build/arch/x86/include/generated/uapi/asm/unistd_32.h` чи
`$build/usr/include/asm/unistd_32.h`.

Той же приклад, синтаксис AT&T

- На жаль, в Юнікс-світі, синтаксис AT&T поширеніший. Швидше за все, ви побачите щось таке:

`_start:`

```
    movl $4, %eax      # __NR_write == 4 -- запис у файл
    movl $1, %ebx      # Дескриптор для stdout
    movl $msg, %ecx     # Вказівник на буфер
    movl $msg_size, %edx # Розмір буфера
    int $0x80          # make syscall

    movl $1, %eax      # __NR_exit == 1 -- exit
    movl $0, %ebx      # Код завершення 0
    int $0x80          # make syscall
```

- Порядок аргументів, в цілому, зворотній до Intel, різні поняття позначають певними символами – \$, % і т.д., детальніше див. відповідну презентацію.

Приклад – Linux, x86-64, SYSCALL, синтаксис Intel, діалект GNU as

```
.intel_syntax noprefix # Same as: -msyntax=intel -mnaked-reg
.global _start
```

```
.text
```

```
_start:
    mov eax,1                # __NR_write (sys_write) == 1
    mov edi,1                # Дескриптор для stdout
    mov rsi, OFFSET msg      # Non-PIC код
    mov edx, OFFSET msg_size
    syscall

    mov eax,60               # __NR_exit (sys_exit) == 60
    xor edi,edi              # exit code 0
    syscall
```

```
.data
```

```
msg:
.ascii "Hello 64-bit Linux world!\n"
msg_size = . - msg
```

- Ідея та ж, хоча порядок аргументів та номери викликів інші. ▶

- Номери системних викликів:

```
#define __NR_write 1  
#define __NR_exit 60
```

- Прототип системного виклику той же, його номер – в RAX, результат – в RAX та (за потреби) RDX.
- Аргументи передаються в такому порядку: RDI, RSI, RDX, R10, R8, R9.
- Номери системних викликів – в `/usr/src/linux/include/asm-x86_64/unistd.h` або `$build/arch/x86/include/generated/uapi/asm/unistd_64.h`.

Той же приклад, синтаксис AT&T

- `_start:`
 `movq $1, %rax` `# __NR_write (sys_write) == 1`
 `movq $1, %rdi` `# Дескриптор stdout -- 1`
 `movq $msg, %rsi` `# Буфер`
 `movq $msg_size, %rdx` `# Його розмір`
 `syscall` `# make syscall`

 `movq $60, %rax` `# __NR_exit (sys_exit) == 60`
 `movq $0, %rdi` `# Код завершення -- 0`
 `syscall` `# make syscall`

POSIX

- Поверх системних викликів широко використовуються обгортки, які спрощують їх використання.
- В POSIX світі це `libc`, стандартна бібліотека C.
- Приклад: системний виклик `sys_open` – відкрити файл. Для 64-бітного Linux⁹, слід передати в `RAX` `0x02`, `RDI` – C-стрічку із іменем, в `RSI` – прапорці, в `RDX` – режим.
- На практиці, із C та більш високорівневих мов, звертання до нього відбуватиметься викликом функції `open()`, оголошеної в `unistd.h`¹⁰ – хедері із основною частиною POSIX API:

```
int open(const char *pathname, int flags);  
int open(const char *pathname, int flags, mode_t mode);
```

- Всередині цієї функції відбуваються мінімальні перевірки аргументів, за потреби, встановлюється `errno`, або відбувається системний виклик. Вони – дуже тонка обгортка над викликом.
- Прикладні програми значно частіше будуть використовувати дещо більш високорівневу функцію стандартної бібліотеки C, `fopen()`¹¹ із `stdio.h`.

⁹В принципі, подробиці можуть змінюватися. Вказано згідно <https://syscalls64.paolostivanin.com>.

¹⁰<http://man7.org/linux/man-pages/man2/open.2.html>

¹¹<http://man7.org/linux/man-pages/man3/fopen.3.html>

Приклад — write та exit в Linux, C

- Приклад за мотивами https://en.wikibooks.org/wiki/X86_Assembly/Interfacing_with_Linux.
- Функції-обгортки із glibc для одноіменних системних викликів:

```
ssize_t write(int fd, const void *buf, size_t count);  
void _exit(int status);
```

- Використання:

```
#include <unistd.h>
```

```
int main(int argc, char *argv[])  
{  
    write(1, "Hello World\n", 12); /* "Hello World" to stdout */  
    _exit(0);                      /* exit, error code 0 (no error) */  
}
```

- Наприклад, реалізація в glibc, хоча і розмазана по файлах, прямолінійна. Із <https://code.woboq.org/userspace/glibc/sysdeps/unix/sysv/linux/open64.c.html>:

```
int __libc_open64 (const char *file, int oflag, ...) {
    int mode = 0;
    if (__OPEN_NEEDS_MODE (oflag)) {
        va_list arg;
        va_start (arg, oflag);
        mode = va_arg (arg, int);
        va_end (arg);
    }
    return SYSCALL_CANCEL (openat, AT_FDCWD, file,
        oflag | EXTRA_OPEN_FLAGS, mode);
}
.....
weak_alias (__libc_open64, open) // Магія weak-символів,
// __libc_open64 та open тепер синоніми, якщо не дано іншої реалізації open
```

- Макрос SYSCALL_CANCEL є в <https://code.woboq.org/userspace/glibc/sysdeps/unix/sysdep.h.html>, фактично, лише викликає макрос INLINE_SYSCALL_CALL із переданими аргументами (див. <https://sourceware.org/glibc/wiki/SyscallWrappers>).
- INLINE_SYSCALL_CALL, в https://code.woboq.org/userspace/glibc/sysdeps/unix/sysv/linux/x86_64/sysdep.h.html, викликає макрос internal_syscallX, де X – кількість аргументів.

- **#define** internal_syscall3(number, err, arg1, arg2, arg3)
{
 unsigned long int resultvar;
 TYPEFY (arg3, __arg3) = ARGIFY (arg3);
 TYPEFY (arg2, __arg2) = ARGIFY (arg2);
 TYPEFY (arg1, __arg1) = ARGIFY (arg1);
 register TYPEFY (arg3, _a3) asm ("rdx") = __arg3;
 register TYPEFY (arg2, _a2) asm ("rsi") = __arg2;
 register TYPEFY (arg1, _a1) asm ("rdi") = __arg1;
 asm **volatile** (
 "syscall\n\t"
 : "=a" (resultvar)
 : "0" (number), "r" (_a1), "r" (_a2), "r" (_a3)
 : "memory", REGISTERS_CLOBBERED_BY_SYSCALL);
 (**long int**) resultvar;
}

Хоча тут теж купа макросів¹², ідея – зрозуміла, якщо знати про inline-асемблер gcc (див. далі).

- Ремарка: код відповідних функцій генерується автоматично, див. відповідь тут: <https://stackoverflow.com/questions/6515583/where-are-syscalls-located-in-glibc-source>.

¹²Які виконують чисто синтаксичні фокуси – див. код.

- `fopen` трохи складніша – вона готує структуру `FILE` (в `glibc` – синонім до `_IO_FILE`, із https://code.woboq.org/userspace/glibc/libio/bits/types/struct_FILE.h.html),

фактична реалізація – із файлу

<https://code.woboq.org/userspace/glibc/libio/iofopen.c.html>, має ім'я `__fopen_internal()`, відносно складна, але рано чи пізно викликає щось типу `_IO_file_open`

(<https://code.woboq.org/userspace/glibc/libio/fileops.c.html>), яка здійснює системний виклик:

```
fdesc = __open_nocancel (filename,
                        posix_mode | (is32not64 ? 0 : O_LARGEFILE), prot);
```

- Для ілюстрації, фрагмент `FILE`

```
struct _IO_FILE
{
    int _flags;                /* High-order word is _IO_MAGIC; rest is flags. */
    /* The following pointers correspond to the C++ streambuf protocol. */
    char *_IO_read_ptr;        /* Current read pointer */
    char *_IO_read_end;        /* End of get area. */
    char *_IO_read_base;       /* Start of putback+get area. */
    char *_IO_write_base;      /* Start of put area. */
    char *_IO_write_ptr;       /* Current put pointer. */
    char *_IO_write_end;       /* End of put area. */
    .....
    int _fileno;
    int _flags2;
```

- В newlib, – компактній реалізації libc¹³, все простіше, див. newlib/libc/stdio/fopen.c¹⁴, newlib/libc/ syscalls/sysopen.c¹⁵ та newlib/libc/include/sys/reent.h¹⁶.

¹³Документація: <https://sourceware.org/newlib/libc.html#fopen>.

¹⁴<https://sourceware.org/git/?p=newlib-cygwin.git;a=blob;f=newlib/libc/stdio/fopen.c;h=1ac551fed2b307222017d307b12d446bbf319ede;hb=HEAD>

¹⁵<https://sourceware.org/git/?p=newlib-cygwin.git;a=blob;f=newlib/libc/syscalls/sysopen.c;h=bc8d9d59f866eb9e4765d6cd669a2f3e08060c30;hb=HEAD>

¹⁶<https://sourceware.org/git/?p=newlib-cygwin.git;a=blob;f=newlib/libc/include/sys/reent.h;h=74b70e9c0914b50a340ddd935303e045ff21d6b5;hb=HEAD>

Приклад — Hello World для DOS 1.0 1981

```
.model small
.stack 100h
.data
    msg db "Hello, world!$"
.code
.startup
    mov dx,offset msg
    mov ah,09h
    int 21h

    ret
```

Коли бачу такий код, на очі навертаються сльози. Із двох причин:

- Починав із чогось схожого, див.
<http://indrekis2.blogspot.com/2013/02/dos-fcb.html> і посилання на попередні пости там.
- З іншого боку, в деяких ВНЗ, на профільних IT-спеціальностях, такий код вивчали, як мінімум, ще в 2018!
- Еволюція диспетчера int 21h: "[Evolution of the DOS INT 21h function dispatcher](#)", код для MS DOS 2.0:
<https://github.com/microsoft/MS-DOS/blob/master/v2.0/source/MSCODE.ASM>.

Древній MS-DOS, int 21h, сучасний FASM

format MZ

```
push    cs
pop      ds

mov     ah, 09h
mov     dx, hello
int     21h

mov     ax, 4C00h
int     21h
```

hello **db** 'Hello world!',24h

- Аргументи системному виклику передаються за допомогою регістрів. Регістри можуть містити адреси блоків пам'яті.
- Код операції – в АН.
- Про помилку повідомлялося встановленням CF, за потреби модифікує інші регістри.

MS Windows

- API Windows організовано складніше.
- Системні виклики прикладними програмістами практично не використовуються – вони нестабільні, навіть їх номери змінюються між версіями, зокрема – багато разів у різних випусках Windows 10.
- Більшість із них реалізовані в ядрі, `ntoskrnl.exe`. Частина, що стосується GUI – в `win32k.sys` (як для 32-бітових, так і для 64-бітових версій).
- Поверх них реалізовано **Native API** – теж практично не документоване, але стабільне. Відповідні функції-обгортки знаходяться в `ntdll.dll`. Це приблизний аналог функцій, що в POSIX знаходяться в `unistd.h`.
- GUI-частина Native API доступна через `user32.dll` і `gdi32.dll`.
- Функції діляться на декілька груп. Особливо цікаві функції із суфіксами `Nt` та `Zw`, де функції із суфіксом `Nt` викликають kernel-space реалізацією функцій із суфіксом `Zw`. Префікс `Rtl` означає частину C Runtime. Більший список див. https://en.wikipedia.org/wiki/Native_API, однак подробиці там зараз (2025) дивно описані.
- ◇ Це загальна практика – в привілейованому коді мати окремий набір викликів/функцій, відмінний від користувацьких API. Ілюстрація для Linux: `kmalloc()`, `printk()`.

- Поверх Native API реалізуються **підсистеми**.
- Найважливіша із них – Win32 API, що складається із KERNEL32.DLL, USER32.DLL, GDI32.DLL і т.д., а для виконання відповідних операцій, окрім засобів GUI, може звертатися до Client Server Runtime Subsystem, csrss.exe, див. https://en.wikipedia.org/wiki/Client/Server_Runtime_Subsystem.
- ◇ В 90-х Windows NT підтримував також OS/2 і POSIX підсистеми. Хоча потім вони були викинуті, завдяки відповідному механізму, ми зараз легко можемо мати **WSL-підсистему**, Windows Subsystem for Linux, https://en.wikipedia.org/wiki/Windows_Subsystem_for_Linux.
- ◇ WSL 2 реалізована по-іншому – це досить типова віртуальна машина.
- Тобто, до прикладу, WinAPI функція CreateFile() рано чи пізно викличе NtCreateFile(), яка через SYSENTER (давніші версії використовували int 2Eh) зайде в ядро, після перевірки там аргументів, викличе ZwCreateFile(). Драйвери останню можуть викликати безпосередньо.
- Простенький приклад: <https://baykara.medium.com/from-top-to-bottom-on-windows-nt-3bd9ea10e7b1> – від exit(42); до syscall.

- MS Windows не є Open Source (хоча, джерельні тексти, все ж, доступні – як через академічні програми, так і через витікання коду даної ОС), тому наводити конкретні приклади реалізації важче.
- "Windows NT(2000) Native API Reference", Gary Nebbett, 1999, наводить такий приклад:

```
HANDLE Win32RootDirectory() {  
    static HANDLE Handle;  
    if (Handle == 0) {  
        NT::UNICODE_STRING ObjectName;  
        NT::RtlInitUnicodeString(&ObjectName, L"\\BaseNamedObjects");  
  
        NT::OBJECT_ATTRIBUTES ObjectAttr = {sizeof ObjectAttr, 0,  
                                             &ObjectName,  
                                             OBJ_CASE_INSENSITIVE};  
  
        ACCESS_MASK DesiredAccess = READ_CONTROL | DIRECTORY_QUERY  
                                     | DIRECTORY_TRAVERSE  
                                     | DIRECTORY_CREATE_OBJECT  
                                     | DIRECTORY_CREATE_SUBDIRECTORY;  
  
        NT::ZwOpenDirectoryObject(&Handle, DesiredAccess, &ObjectAttr);  
    }  
    return Handle;  
}
```

- "Reversing: Secrets of Reverse Engineering", Eldad Eilam, 2005, наводить приклади реалізації Native API функцій у user-space.

- Windows 2000, int 2Eh інтерфейс:

```
ntdll!ZwReadFile:
```

```
77f8c552 mov eax,0xa1 ; Номер системного виклику
```

```
77f8c557 lea edx,[esp+0x4] ; Перший аргумент
```

```
77f8c55b int 2e
```

```
77f8c55d ret 0x24
```

- Windows XP, за допомогою sysenter – двоетапний, оскільки цей метод не зберігає контексту:

```
ntdll!ZwReadFile:
```

```
77f4302f mov eax,0xbf
```

```
77f43034 mov edx,0x7ffe0300
```

```
77f43039 call edx
```

```
77f4303b ret 0x24
```

За адресою 0x7ffe0300:

```
SharedUserData!SystemCallStub:
```

```
7ffe0300 mov edx,esp
```

```
7ffe0302 sysenter
```

```
7ffe0304 ret
```

Ядро повертається відразу в ZwReadFile, підглядаючи адресу в стеку, ret із SystemCallStub не буде використано.

Посилання

- https://en.wikipedia.org/wiki/System_call.
- Native API документований слабо. Дещо можна довідатися тут:
<http://www.osronline.com/article.cfm?article=257>,
<http://www.osronline.com/article.cfm?article=91> та
<http://www.informit.com/articles/article.aspx?p=22442>.
- Дещо застаріле, враховуючи нестабільність Native API, але все ще актуальне джерело: "Windows NT/ 2000 Native API Reference", Gary Nebbett, 1999. (В Інтернеті можна знайти книгу без вступу, вступ доступний на Google Books).
- <https://web.archive.org/web/20121224002314/http://netcode.cz/img/83/nativeapi.html> – "Inside the Native API", Mark Russinovich.
- Таблиця системних викликів, за версіями MS Windows:
<https://github.com/j00ru/windows-syscalls>.
- Про нове повернення int 2E (у зв'язку із віртуалізацією, здається):
blog.amosys.fr/windows10_TH2_int2E_mystery.html.
- Загальний опис впорядкування рівнів абстракції в Windows Vista: <https://blogs.msdn.microsoft.com/larryosterman/2005/08/23/larry-goes-to-layer-court/>.
- Про системні виклики Linux: <https://blog.packagecloud.io/eng/2016/04/05/the-definitive-guide-to-linux-system-calls/>

- Ще одне лаконічне джерело:
https://stackoverflow.com/questions/2535989/what-are-the-calling-conventions-for-unix-linux-system-calls-on-__kernel_vsyscall
– зокрема, розглянуто virtual Dynamic Shared Object (vDSO) та
- ◇ Аналог для Windows: <https://devblogs.microsoft.com/oldnewthing/20221216-00/?p=107598>,
"Why doesn't Windows use the 64-bit virtual address space below 0x00000000'7ffe0000?".
- http://blog.rchapman.org/posts/Linux_System_Call_Table_for_x86_64/ – список.
- Кілька списків: <https://syscalls64.paolostivanin.com>,
<http://www.cpu2.net/linuxabi.html>.
- <https://habr.com/ru/post/347596/> – "Эволюция системных вызовов архитектуры x86", показано приклади виконання системних викликів за допомогою різних механізмів, `int 80h`, `sysenter/sysexit`, `syscall/sysret`, `vsyscall`, `vDSO`

vDSO

- Системні виклики – достатньо повільні.
- ◊ Пара сотень тактів на виклик та повернення і ще порядку тисячі в коді ядра для найпростіших операцій.
- Тому, для частих, але недорогих операцій, механізм виклику домінуватиме у затратах часу програми. Цей процес хотілося б оптимізувати.
- Для сучасних Linux таким механізмом є vDSO – virtual (ELF) dynamic shared object.
- Ідея наступна: код read-only викликів та дані для них відображаються в користувацький адресний простір.
- З точки зору процесу це виглядає як динамічна бібліотека (linux-vdso.so.1 для x86-64, linux-gate.so.1 – x86-32):

```
indrekis@DESKTOP-21UTCOT:~$ ldd /bin/ls
linux-vdso.so.1 (0x00007ffffbf51000) <===== відповідного файлу немає
libselinux.so.1 => /lib/x86_64-linux-gnu/libselinux.so.1 (0x00007f3095c20000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f3095a20000)
.....
```

- Приклад так реалізованих викликів: `gettimeofday()` та `clock_gettime()`.

- `gettimeofday()` викличе `__vdso_gettimeofday` – код системного виклику, який виконуватиметься в user space.
- Ще однією важливою частиною vDSO є `__kernel_vsyscall` – механізм, який відображається в кожен процес та інкапсулює технічні подробиці виконання системного виклику – чи це `int 80h`, `syscall`, `sysenter` чи щось інше.
- На відміну від старішого аналогічного механізму, `vsyscall`, vDSO не має проблем із Address space layout randomization (ASLR).
- `glibc` інкапсулює використання vDSO.
- Вручну звертання виглядало б якимось так:

```
int (*f)(struct timeval *, struct timezone *);  
struct timeval tm = {0};
```

```
void *vdso = dlopen("linux-vdso.so.1",  
                  RTLD_LAZY | RTLD_LOCAL | RTLD_NOLOAD);  
assert(vdso && "vdso not found");
```

```
f = dlsym(vdso, "__vdso_gettimeofday");  
assert(f);  
f(&tm, NULL);
```

- Подробиці: <https://lwn.net/Articles/604515/>,
<https://blog.packagecloud.io/eng/2016/04/05/>

- Порівняння продуктивності системних викликів, виконаних різними механізмами із статті

<https://habr.com/ru/post/347596/>:

Реализация	час (нс)
int 80h	498
sysenter	338
syscall	278
vsyscall emulate	692
vsyscall native	278
vDSO	37
vDSO-32	51

Також в цій статті є втрати на захист від Meltdown, "Kernel page-table isolation"¹⁷ – 50-170%, в залежності від форми виклику, хіба для vDSO – 0.

¹⁷https://en.wikipedia.org/wiki/Kernel_page-table_isolation

- Подобиці vDSO:
- <https://lwn.net/Articles/604515/>,
- <https://blog.packagecloud.io/eng/2016/04/05/the-definitive-guide-to-linux-system-calls/#virtual-system-calls>
- <http://man7.org/linux/man-pages/man7/vdso.7.html>,
<https://stackoverflow.com/a/19942352>.
- Як створити свій vDSO: <https://www.linuxjournal.com/content/creating-vdso-colonels-other-chicken?page=0,0>
- "What makes cheap_steady_clock faster than std::chrono::high_resolution_clock?"
- ◇ "Why doesn't Windows use the 64-bit virtual address space below 0x00000000'7ffe0000?"

Види привілей CPU: на практиці – як завжди...

- На практиці із ідеальною моделлю User/Supervisor є ряд нюансів. Розглянемо їх на прикладі x86.
- Цей процесор базово підтримує 4 кола захисту: ring 0, ring 1, ring 2, ring 3.
- Сучасні OS зазвичай використовують лише ring 0 як kernel mode, та ring 3 – user mode.
- ◊ OS/2 використовувала ring 2 для користувацьких програм із правом доступу до вводу-виводу.
- ◊ Через проблемність віртуалізації x86, VMWare використовувала ring 1 для ядра гостьової ОС, а сама працює на ring 0. Не знаю, наскільки зараз це практикують.
- Якщо доступ до CR0 чи команда LIDT є лише на ring 0, чи буде доступ до I/O визначають біти 12 і 13 в регістрі прапорців EFLAGS/RFLAGS, так званий IOPL¹⁸.
- ◊ Наприклад, команди CLI та STI – заборонити та дозволити переривання, доступні на колі захисту, починаючи із вказаного в IOPL і нижче.
- ◊ Зрозуміло, що OS/2 використовували $\text{IOPL} = 10b = 2$.

¹⁸Він також є однією з причин проблем із віртуалізацією. PUSHF і POPF мовчки поводяться по-різному із цими прапорцями на ring 0 і на решті рівнів.

- Під ring 0 x86 є ще рівні.
- Умовно, ring -1 – рівень гіпервізора, що з'явився із появою засобів підтримки віртуалізації, Intel VT-x (2005) чи AMD-V (2006). Ці розширення додали команди, доступні лише на -1 рівні.
- Помітно раніше, із 386SL (1990) з'явився режим System Management Mode (SMM), який ретроспективно назвали ring -2. Призначався для різних технічних речей – ініціалізації чіпсету, аварійного вимикання внаслідок перегріву, емуляції пристроїв¹⁹ тощо. Вхід внаслідок переривання на спеціальному піні, SMI#. Невидимий для ОС. Тому руткити та віруси себе там добре почувають.
- Ще нижче знаходяться засоби керування материнською платою²⁰. В Intel це Intel Management Engine (IME). Оскільки IME має доступ до всього в системі взагалі і повністю прозоро для ОС чи й SMM, іноді її називають ring -3. Але вона – поза процесором.
 - ◇ IME працює весь час, поки на материнській платі є живлення. CPU – варіант x86, ОС – MINIX 3. Має доступ до мережі. Цінна мішень для хакерів – див. вразливості, https://en.wikipedia.org/wiki/Intel_Management_Engine.
 - ◇ Аналог від AMD: AMD Platform Security Processor (PSP).

¹⁹Саме так USB клавіатури та мишки доступні до завантаження ОС, так званий USB legacy support.

²⁰Це не термін, устаєного терміну мені не траплялося.

Інші системи

- ARM v7 одразу мав три рівні – user, supervisor і hypervisor.
- OpenVMS на Alpha AXP – використовує 4 рівні: user, supervisor (Command language interpreter – DCL), executive (RMS – record management services), kernel (системні сервіси, pager, scheduler, драйвери).
- Кажуть, Multics мала 8 кіл:
https://en.wikipedia.org/wiki/Protection_ring.

strace

- Утиліта відстеження системних викликів.

pinsyscalls – OpenBSD

- В OpenBSD 7.5 (2024) є спеціальний системний виклик, pinsyscalls, який дозволяє робити системні виклики лише зі вказаного інтервалу адрес:
`https://man.openbsd.org/pinsyscalls.2`, `https://www.undeadly.org/cgi?action=article;sid=20240129060150`.
- ◇ Так можна дозволити робити їх лише libc, але не прикладному коду.

